

1 – System Design Assumptions

Status

Accepted

Context

The system is designed for event environments where internet connectivity may be limited or unavailable. It should be easy to use, quick to deploy, and not rely on centralized infrastructure.

Decision

We adopt the following architectural principles:

- No central backend
- Tokens are self-contained and independently verifiable
- Data exchange is handled via QR codes
- Authenticity is ensured using asymmetric cryptography
- The application is delivered as a mobile-first web app

Consequences

- + Works offline and in constrained environments
 - + Minimal infrastructure required
 - No global system state or real-time tracking
 - Preventing misuse becomes more challenging
-

2 – Token generation, distribution and verification

Status

Accepted

Context

We need a clear and deterministic flow defining how tokens are created, signed, distributed, and redeemed.

Decision

The system follows this process:

1. A supervisor generates a key pair for the trader.
2. The private key along with the trader's ID is distributed to the authorized trader.
3. The trader creates a token containing the point value, trader ID and unique ID.
4. The token is signed using the trader's private key.
5. The signed token along with the plain point value and trader ID are encoded into QR.
6. The trader transfers tokens by presenting QR codes to customers.
7. Customers can share tokens between themselves or transfer them to the supervisor.
8. Tokens are verified using the public key that only the supervisor has a hold of.
9. Used tokens are added to the supervisor block list so that they can't be reused.

Consequences

- + Simple and predictable flow
 - + No synchronization required between devices
 - + No possibility of duping (tokens are unique and can only be made by traders)
-

3 – Token Transfer via QR Codes

Status

Accepted

Context

Token transfer between players must be secure, verifiable, and compatible with limited infrastructure. To facilitate medium-sized to large transfers while keeping each QR scannable, a multi-QR sequence is required.

Decision

We define the token transfer as a sequence of QR codes with the following structure:

1. **Header QR**
 - Contains the customer ID
 - Total number of QR codes in the sequence
 - Total points being transferred
2. **Data QR Codes**
 - Contain token data (signed token, plain point value and trader ID)
3. **Footer QR (Transaction Flag)**
 - Confirms the end of the sequence
 - Allows the receiver to verify that the transfer is complete

Consequences

- + Makes larger transfers easier
 - + Allows verification of transaction integrity at each step
 - Requires sequential scanning; skipping a QR breaks the sequence
 - Slightly higher user effort compared to a single QR transfer
-

4 – Technology Stack Selection

Status

Proposed

Context

We considered both a multiplatform approach with kotlin and a traditional web stack for implementing the client application.

Decision

We are leaning into **TypeScript and React** over Kotlin Multiplatform.

Rationale

- Faster to develop and iterate
- Strong ecosystem for QR handling and cryptography
- Runs on any modern device without installation

Consequences

- + Rapid development and easy deployment
 - + Broad compatibility across devices
-

5 – Cryptography Choice

Status

Accepted

Context

Tokens must be tamper-proof while remaining compact enough to fit inside QR codes.

Decision

We use **EdDSA with Ed25519** for signing tokens.

Rationale

- Compact signature size
- Strong, modern security guarantees
- Efficient verification

Consequences

- + Well-suited for QR-based data transfer
 - + High performance and security
 - EdDSA is deterministic - to achieve unique tokens the payload needs to be randomized with unique ID's
-

6 – QR Code Standard

Status

Accepted

Context

The QR code must store both token data and its signature while remaining scannable under typical conditions.

Decision

We use up to **QR Code Version 12** with **Low** error correction as a balance between capacity and usability.

Consequences

- + Enough space for three signed tokens
 - + Faster scanning than higher QR versions
 - + Works with standard QR scanners
 - Low error correction is more prone to errors
-

7 – Token signature uniqueness

Status

Accepted

Context

Each token signature must be unique to mitigate the risk of collisions and support validation logic.

Decision

We also use **UUIDs (version 4)** to randomize the token signatures.

Rationale

- Practically collision-free
- Easy to generate without coordination

Consequences

- + Reliable uniqueness
 - + No central authority required
-

8 – Customer name uniqueness

Status

Accepted

Context

Each customer name must be unique to mitigate the risk of collisions and support validation logic.

Decision

We use **UUIDs (version 4)** to randomise the customer names.

Rationale

- Practically collision-free
- Easy to generate without coordination

Consequences

- + Reliable uniqueness
 - + No central authority required
 - Adds overhead to QR payload size
-